

EMS Arena

One-page summary based on repo evidence only

What it is

EMS Arena is a Django-based educational management system for universities and other educational institutions. The repo combines course delivery, exams, assignments, labs, blog content, and a live quiz mode in one server-rendered web app.

Who it's for

Primary persona: teachers and academic staff who manage courses, exams, assignments, and labs. Students are secondary end users for enrollment, submissions, exams, and live sessions.

What it does

- Course hubs with topics, resources, enrollments, and groups.
- Test and written exams, question banks, randomization, time limits, and anonymous grading.
- Assignments and projects with deadlines, retries, file or link submission, and teacher feedback.
- Lab workflows with per-student question allocation, timed blocks, and grading.
- Live exam sessions with QR or PIN join, leaderboards, and real-time host and player updates.
- Blog, subscriptions, notifications, organization roles, audit logging, and multi-language UI.

How it works

Browser/UI

Templates, Bootstrap 5, vanilla JS, AJAX

Django app

config.urls mounts blog, live_exam, courses, assignments, labs, more

Data/storage

PostgreSQL by default; local SQLite fallback; protected media files

Async/realtime

ASGI + Channels sockets; Redis cache/channel layer; Celery email tasks

- One ASGI entrypoint serves both HTTP and WebSocket traffic (`config/asgi.py`).
- Versioned live-exam API routes live under `/api/v1/`; domain apps mount from `config/urls.py`.
- Local settings can run with SQLite, in-memory cache/channel layers, and console email when external services are absent.

How to run

- Create and activate a virtualenv: `python3 -m venv venv && source venv/bin/activate`
- Install deps: `pip install -r requirements.txt`
- Create `.env` with at least `SECRET_KEY=...`; `.env.example`: Not found in repo.
- Apply DB setup: `python manage.py migrate` and optionally `python manage.py createsuperuser`
- Start the app: `python manage.py runserver`

Optional full stack: `docker compose up -d` for Postgres and Redis. Local settings also support SQLite and in-memory defaults.

Repo evidence used: README.md, config/settings/base.py, config/settings/local.py, config/urls.py, config/asgi.py, config/celery.py, apps/live_exam/consumers.py, core/email_tasks.py.